START HERE

Deploy SJC using your browser

Client demonstration guide | Manual upload + MySQL | Version 1.0

Use Hostinger hPanel, File Manager and phpMyAdmin to install a **prepared SJC release**. The client does not need Terminal, SSH or local build tools.

Choose PHP/HTML Website
SJC runs on Laravel 13 and PHP 8.4. Its JavaScript builds the page assets. A hosting plan may offer Node.js apps, but SJC must be added as a PHP/HTML website, not as a Node.js web app.

Have these items before the demonstration

Supplied by the developer	Purpose
Application ZIP	Laravel code with installed production PHP dependencies and private runtime folders.
Public-files ZIP	Ready-built CSS/JavaScript, images, .htaccess and a correctly configured index.php.
MySQL .sql file	Tested schema, baseline settings/roles and approved demo or client data.
Private handover details	Environment template, application key, admin access and exact cron entries.

Archive names in this guide are examples: **SJC_application.zip**, **SJC_public.zip** and **SJC_mysql.sql**. Confirm the actual supplied filenames with the developer.

Readiness check
The reviewed source has MySQL configuration, but the documented live SJC site uses SQLite. This guide does not establish that a MySQL release has been built or tested. Get the tested ZIPs and SQL file before following the client steps.

Demo sequence: create PHP site → set PHP → upload both ZIPs → create/import MySQL → edit .env → configure cron and mail → test.

STEPS 1-2

Create the PHP website

Use a new demo domain or subdomain first. Menu labels can vary between Hostinger accounts.

Hostinger dashboard > Websites > Create site > PHP/HTML website

1. Sign in to the client's Hostinger account. Select the hosting plan that will run SJC.

2. Choose **Create site** (sometimes **Add website**), then select **PHP/HTML website**. Enter the client's domain or demo subdomain and complete setup. [R1]

3. Open that website's **Manage site** dashboard. Check the selected domain before changing settings.

If you only see the Node.js app screen
Return to Websites and choose the PHP/HTML website option. If that option is unavailable on the client's account, ask Hostinger to confirm PHP hosting availability before continuing. Adding a Node.js start file will not turn this Laravel code into a Node.js backend.

Select the PHP runtime

Manage site > search for PHP Configuration > PHP version > Update

4. Select **PHP 8.4**. SJC requires at least PHP 8.4.1; Hostinger handles the patch release. This setting can affect other PHP websites on the same plan, so check them before applying it. [R2]

5. Have the developer verify required extensions, particularly **pdo_mysql**, mbstring, DOM/XML, GD, intl, ZIP, bcmath and fileinfo. Some are built in rather than separate toggles.

What you should see before moving on

The new domain appears under Websites. Its dashboard provides File Manager, PHP Configuration and database management. PHP 8.4 is selected.

This walkthrough covers installing a new prepared release. For an existing populated website, use the developer-approved update and backup plan on page 8 instead of importing a fresh demo database.

STEPS 3-4

Upload the website files

The prepared archives must be designed for the two-folder layout below.

Manage site > Files / File Manager > website files

1. Open File Manager. Go to the website directory that contains **public_html**. If it opens inside public_html, go up one level. [R3]

2. Create a sibling folder named **journal_app**. Open it, upload **SJC_application.zip**, select the archive and choose **Extract**.

3. Open **public_html**. Upload and extract **SJC_public.zip** there. Check that index.php is directly inside public_html, without an extra SJC_public subfolder.

```
domains/yourdomain.com/
  journal_app/
    app/  bootstrap/  config/  routes/
    vendor/  storage/  scripts/  artisan
    .env
  public_html/
    index.php
    .htaccess
    build/manifest.json
    storage/  [prepared public-upload location]
```

Keep the two areas separate
journal_app contains private code, configuration and private documents. public_html contains only public website files. Keep the SQL import file and private configuration backups off public_html.

4. Check that **vendor/autoload.php** exists in journal_app and **build/manifest.json** exists in public_html. Enable hidden-file visibility if .env or .htaccess is not shown.

5. The supplied index.php and application public-path setting must already point to this layout. Ask the developer for corrected ZIPs if they still expect Laravel's default public folder.

If the new site contains a placeholder page, move that file into a private backup location before removing it from the public root. Preserve any existing files until you have confirmed their purpose.

STEPS 5-6

Create and import MySQL

Use a new empty database for the client demonstration.

Manage site > Databases > Management

1. Confirm the selected website. In **Create a New MySQL Database and Database User**, enter a database name, username and strong password, then click **Create**. [R1, R4]

2. Save the complete database name and username exactly as displayed. Hostinger adds an account prefix, such as **u123456789_sjc**. Keep the password in the client's password manager.

Value to record	Example only
Database name	u123456789_sjc
Database username	u123456789_sjcuser
Database password	The password you just created
Host / port	localhost / 3306 for this same-host PHP setup; verify the host details in hPanel.

Your new database > Enter phpMyAdmin > Import

3. Open **Enter phpMyAdmin** for that database. Confirm its name in the left sidebar. Choose **Import**, select the developer's **SJC_mysql.sql**, then click **Import** or **Go**. [R1, R5]

4. Wait for the import-success message. The database should contain tables such as users, roles, settings, articles, migrations, jobs and sessions, according to the supplied release.

A SQLite file is not a MySQL import
Do not select database.sqlite, rename it to .sql, or use a PostgreSQL dump. The developer must supply a MySQL-compatible export and verify its schema/data on the target MySQL or MariaDB version.

If import fails, capture the error text and ask for a corrected export. A partly imported database is not ready for the next step. Do not re-import over a database with real client activity.

STEP 7

Connect SJC to the database

Edit the developer-provided environment template using File Manager.

File Manager > journal_app > .env > Edit > Save

Use the complete supplied template. Update the relevant lines below; this is an **example excerpt**, not a complete environment file. Replace the example domain, prefixed names and password.

```
APP_ENV=production
APP_DEBUG=false
APP_URL=https://journal.example.com
APP_FORCE_HTTPS=true

DB_CONNECTION=mysql
DB_HOST=localhost
DB_PORT=3306
DB_DATABASE=u123456789_sjc
DB_USERNAME=u123456789_sjcuser
DB_PASSWORD="YOUR_DATABASE_PASSWORD"

CACHE_STORE=database
SESSION_DRIVER=database
SESSION_ENCRYPT=true
SESSION_SECURE_COOKIE=true
QUEUE_CONNECTION=database
```

Complete the private handover values

Application key: insert the key securely supplied by the developer. A fresh installation needs its own key. A copy containing existing encrypted data needs the matching key and a developer-approved transfer plan.

Mail and domain settings: use the client's sender mailbox, SMTP details and notification recipients. Ask the developer to update enabled Google sign-in callbacks and domain-dependent analytics/cookie settings for the new domain.

Production template requirements
Local account bypasses must be disabled. The ZIP must not contain cached configuration from another computer or server. If saved .env changes appear to be ignored, have the developer correct the release package; do not expose a public setup or cache-clearing page.

Keep .env inside journal_app. Do not put credentials into the public files ZIP, screenshots or the deployment guide. Keep the complete template's other required settings.

STEP 8

Enable uploads and email jobs

All actions here take place in the browser. Cron strings are pasted into hPanel forms.

Confirm the prepared storage layout

The developer must supply a public-upload mapping that works without the client creating a server symlink. A prepared direct mapping can store public media in **public_html/storage**, while private manuscripts stay in **journal_app/storage/app/private**. Merely copying images once does not configure future uploads.

In File Manager, confirm journal_app/storage and bootstrap/cache are writable by the hosting account. Use the developer's tested permissions. Do not make the entire application writable or set everything to 777.

Set the two background schedules

Manage site > search for Cron Jobs > Create a cron job

Choose **Custom**. Paste the developer-confirmed entry into **Command to Run**, set every schedule field to ***** (every minute), and click **Save**. Repeat for the second entry. Hostinger runs them automatically. [R6]

Entry 1 - scheduled publication and maintenance

```
/opt/alt/php84/usr/bin/php /home/ACCOUNT/domains/DOMAIN/journal_app/artisan schedule:run
```

Entry 2 - guarded mail queue worker

```
/opt/alt/php84/usr/bin/php /home/ACCOUNT/domains/DOMAIN/journal_app/scripts/hostinger-mail-worker.php
```

Replace **ACCOUNT** and **DOMAIN** with this installation's actual folder names. The developer must confirm the PHP 8.4 binary path and that the worker file is included. These are templates to paste in hPanel, not commands for the client to run in Terminal.

Email acceptance check
Configure the client's SMTP credentials first, then use an agreed test address to submit one contact form or request one password reset. Confirm receipt. A green cron status alone does not prove delivery.

If the release deliberately includes a mail-paused flag, have the developer confirm when it may be removed. A fresh demo database should not replay old production queue jobs.

STEP 9

Run the client demonstration

Finish on the intended HTTPS domain, then repeat the main checks on mobile.

Point the domain to this website using the account's domain-connection instructions. Preserve existing email DNS records. In the website dashboard, confirm the SSL certificate is active and HTTPS redirection is enabled.

Show the client	Expected result
1. Home page and an article	Correct SJC content, navigation, styles and images load.
2. /up	Health page responds. This is a liveness check, not a complete database or email test.
3. /admin/login	The supplied client administrator can sign in and see the correct dashboard.
4. Create a demo draft	Save a harmless test article and reopen it; the changes remain.
5. Upload and retrieve a file	Public media appears. A private manuscript remains accessible only through an authorized account.
6. One email workflow	The agreed recipient receives the contact/reset/notification test.
7. Scheduled activity	A controlled scheduled item runs with the cron jobs; inspect the result and cron output.

If something fails

Symptom	First check
Node framework/build error	Create SJC as a PHP/HTML website.
500 error or blank page	PHP version, vendor files, supplied paths, writable runtime folders and private application logs.
Database access denied	DB_HOST, prefixed DB_DATABASE/DB_USERNAME and password in .env.
No styles or missing images	public_html/build/manifest.json, the domain URL and prepared public-upload mapping.
Login/419 or missing mail	HTTPS/session settings and domain cookies; SMTP, both cron jobs and the mail queue for email issues.

Use only agreed demo data. Record the tested domain, date, release version and any unresolved issue in the handover. Remove demo records through the normal approved application workflow.

HANDOVER APPENDIX

Developer readiness and references

This page explains what must be ready before the browser-only demonstration.

Developer checklist - complete before giving the client the ZIPs

1. Prepare the production PHP dependency folder and built frontend assets. Include required package discovery metadata. Exclude source-machine config/route/view caches, local environment files, unrelated backups, logs and queued production messages.

2. Adapt index.php and the Laravel public-path binding for journal_app/public_html in both HTTP and scheduled execution. Configure and test public uploads without a client-side symlink step; preserve private manuscript access.

3. Validate migrations and workflows against the target MySQL/MariaDB server. Export a matching schema with migration history, baseline roles/settings and only approved data. Include an agreed admin-account onboarding path, not a shared hard-coded password.

4. Supply a complete production environment template, a securely delivered application key, exact account/domain paths, two tested cron entries and a short acceptance record. For an existing-data transfer, include consistent private/public uploads and the matching encryption key where required.

Existing website or later updates
Back up the matching database, uploads and private environment first. Preserve the installed application key and client data. Use a tested schema-update plan; do not overwrite an active database with the demo SQL file. A fresh database import is not an update mechanism.

Evidence and limits of this guide

Source reviewed: composer.json and package.json (PHP backend, Vite assets); config/database.php (MySQL option); public/index.php and bootstrap/app.php (public-path setup); config/filesystems.php (storage); README and CI configuration (SQLite/PostgreSQL coverage); docs/MAIL_SETUP.md and the Hostinger worker (background email). No live client installation or MySQL acceptance test was performed for this guide.

Official Hostinger references - checked 01 November 2026

R1 Website creation, file upload and database import

R2 PHP Configuration and version selection

R3 File Manager: upload, extract and edit

R4 Manage MySQL databases

R5 Open phpMyAdmin

R6 Create and review cron jobs

The document illustrates the click sequence; it does not contain screenshots from the client's account. Follow the matching labels in the client's current dashboard.